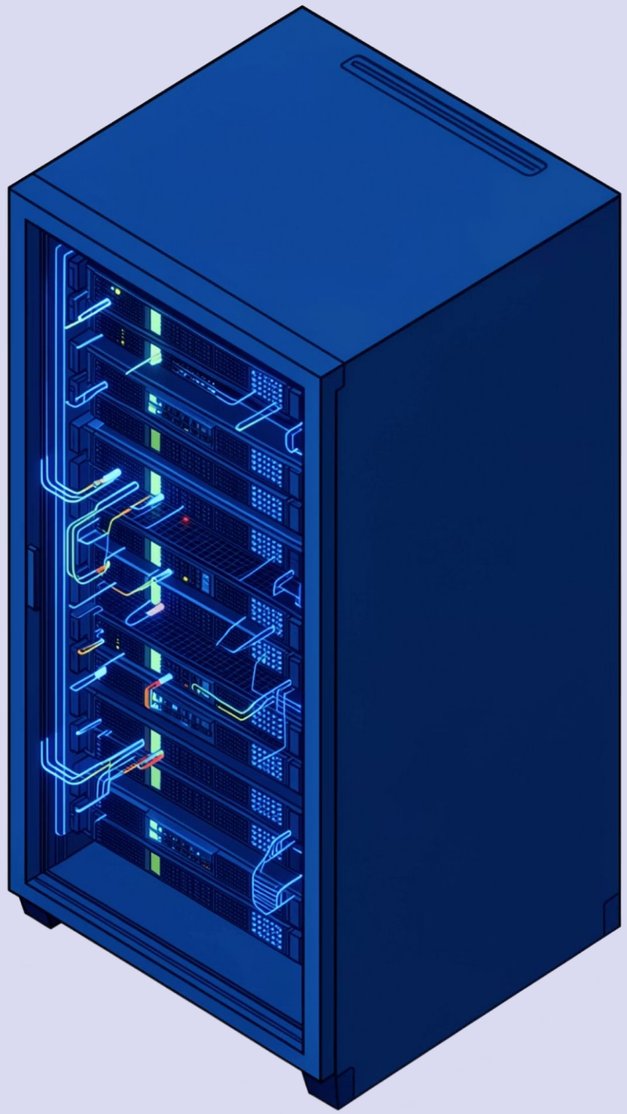
rmproxy

# TLS Reverse Proxy для Astra Linux SE

Аутентификация через Kerberos/SPNEGO, мандатный контроль доступа (MAC) на основе PARSEC, JWT-cookie кэширование и прозрачное проксирование на backend. Ориентирован на окружение Astra Linux / FreeIPA / SSSD / LDAP / PARSEC.

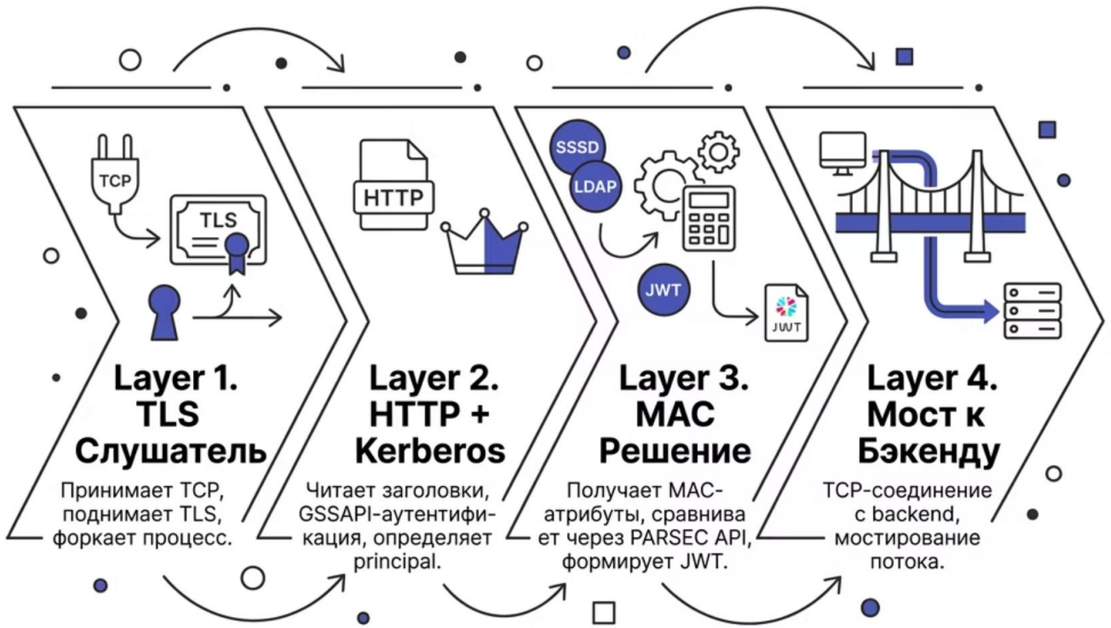


# Структура репозитория

Проект состоит из следующих ключевых файлов:

| Файл | Назначение |
|---|---|
| `.gitignore` | Исключает `cert/`, бинарник `proxy`, `notes.txt` |
| `README.md` | Инструкция по зависимостям, сборке и настройке |
| `mrp.conf` | Runtime-конфиг: порт, backend, LDAP/DC, сертификаты |
| `config.h / config.c` | Структура конфигурации и парсер `key = value` |
| `http.h / http.c` | HTTP/JWT-утилиты: заголовки, GSSAPI, JWT, ответы 401/302/403 |
| `sssd.h / sssd.c` | Чтение атрибутов пользователя из SSSD InfoPipe и хоста из LDAP |
| `proxy.c` | Основной сервер: TLS listener, fork-per-connection, Kerberos-auth, MAC-проверка, проксирование |
| `74x-ald-host-mac.ldif` | LDAP schema extension: атрибут `x-ald-host-mac`, objectClass `x-ald-host-parsec` |

# Архитектура



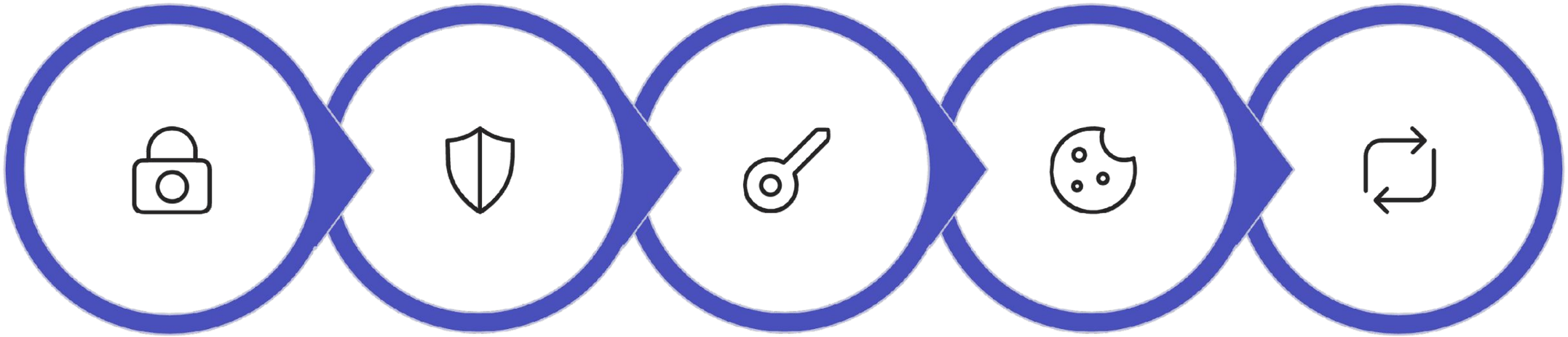

### Модель процесса

Сервер работает по схеме **accept + fork**:

- Родительский процесс слушает входящий порт
- На каждое подключение создаётся дочерний процесс
- Дочерний процесс выполняет весь TLS/auth/proxy pipeline

Завершившиеся процессы подчищаются через `SIGCHLD`

# Поток обработки запроса



**Первый шаг**

**TLS + No JWT**

**401 + GSSAPI**

**MAC → 302 JWT**

**Повторный доступ**

При первом обращении клиент проходит полную цепочку Kerberos-аутентификации и MAC-решения. При повторных запросах прокси валидирует JWT-cookie и принимает решение о проксировании без повторной аутентификации.

# Алгоритм принятия решения по MAC

### Принцип сравнения

Берётся **минимальный** из MAC-уровней пользователя и клиентского хоста — это фактический субъектный уровень. Он сравнивается с MAC-уровнем целевого хоста через PARSEC API.

> Результат сравнения `>=` выставляет `has_access = 1`, иначе `0`.

# Конфигурация

## Формат `mrp.conf`

```
port     = 8888
t_port   = 8000
t_addr   = 0.0.0.0
dc_url   = astraipa.domain.net
cert_path = ./cert/
```

## Параметры

| Параметр | Описание |
|---|---|
| `port` | Порт приёма TLS-клиентов |
| `t_port` | TCP-порт backend-сервиса |
| `t_addr` | IPv4-адрес backend (только literal) |
| `dc_url` | Хост LDAP/DC для bind и lookup |
| `cert_path` | Каталог с `cert.pem` и `key.pem` |

# Сборка

## Команда компиляции

```
gcc proxy.c sssd.c config.c http.c -o proxy \
  -lpdp -lgssapi_krb5 -lcrypto \
  -lsystemd -lldap -lssl
```

## Зависимости

### GSSAPI / Kerberos

`libkrb5-dev`

### OpenSSL

`libssl-dev`

### OpenLDAP

`libldap-dev`

### SSSD InfoPipe

`libsystemd-dev`

### PARSEC MAC API

`parsec-dev`

# Подготовка окружения

## 1

### Сертификаты

Self-signed в `./cert/`:

```
openssl req -x509 -newkey rsa:4096 \
  -keyout cert/key.pem \
  -out cert/cert.pem \
  -days 3650 -nodes
```

## 2

### Kerberos / keytab

Актуальный ticket cache, HTTP keytab, сервисный principal:

```
sudo execaps -c 0x00004 \
  env KRB5_KTNAME=/var/lib/ipa/gssproxy/http.keytab
./proxy
```

## 3

### LDAP schema и IPA

Добавить схему из `74x-ald-host-mac.ldif`, назначить атрибуты хосту, создать permission на чтение.

## 4

### SSSD

В `sssd.conf` включить `x-ald-user-mac` в domain-секции и `[ifp]`:

```
user_attributes = +x-ald-user-mac
```

# HTTP-слой

## Поддерживаемые ответы

### 401 Unauthorized

`WWW-Authenticate: Negotiate`

### 302 Found

`Set-Cookie: jwt=...`

### 403 Forbidden

При `has_access != 1`

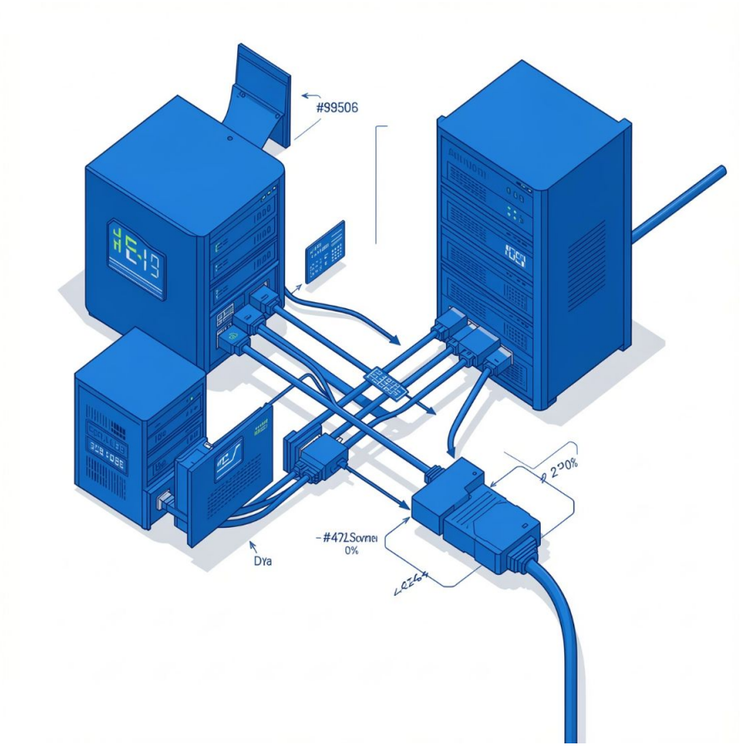
## Структура JWT claims

```
typedef struct {
  char uname[256];
  int  has_access;
  long exp;
} s_jwt;
```

Атрибуты cookie: **HttpOnly**, **Secure**, **SameSite=Strict**, **Path=/**

Модуль `http.c` также умеет: дочитывать заголовки до `\r\n\r\n`, извлекать `Host:` и `Authorization: Negotiate`, кодировать/декодировать Base64, формировать и проверять JWT с HMAC-SHA256.

# Backend Bridge




## Режим работы

После `has_access == 1` прокси:

Создаёт TCP-сокет и подключается к `t_addr:t_port`

Передаёт сохранённый `prefetch_req`

Работает в режиме **прозрачного мостирования**: TLS client ↔ backend socket

## Ограничения

Upstream к backend — **без TLS**

- Backend доступен только по IPv4 literal
- Нет HTTP-aware rewriting заголовков

Это **stream bridge**, а не полнофункциональный L7 reverse proxy